DESPLIEGUE DE APLICACIONES WEB — CURSO 2026/2027

Docker Compose

Tema 2 — Virtualización y contenedores



Objetivo de la sesión

Punto de partida

Las imágenes de app y bd ya están publicadas.

El despliegue requiere recordar varias órdenes y sus parámetros.

Resultado

Un compose.yaml con servicios, red y persistencia.

Configuración y secretos separados.

Arranque condicionado por la disponibilidad de PostgreSQL.

El repositorio describe cómo ejecutar el conjunto y cómo preparar sus archivos locales.

De órdenes a una descripción declarativa

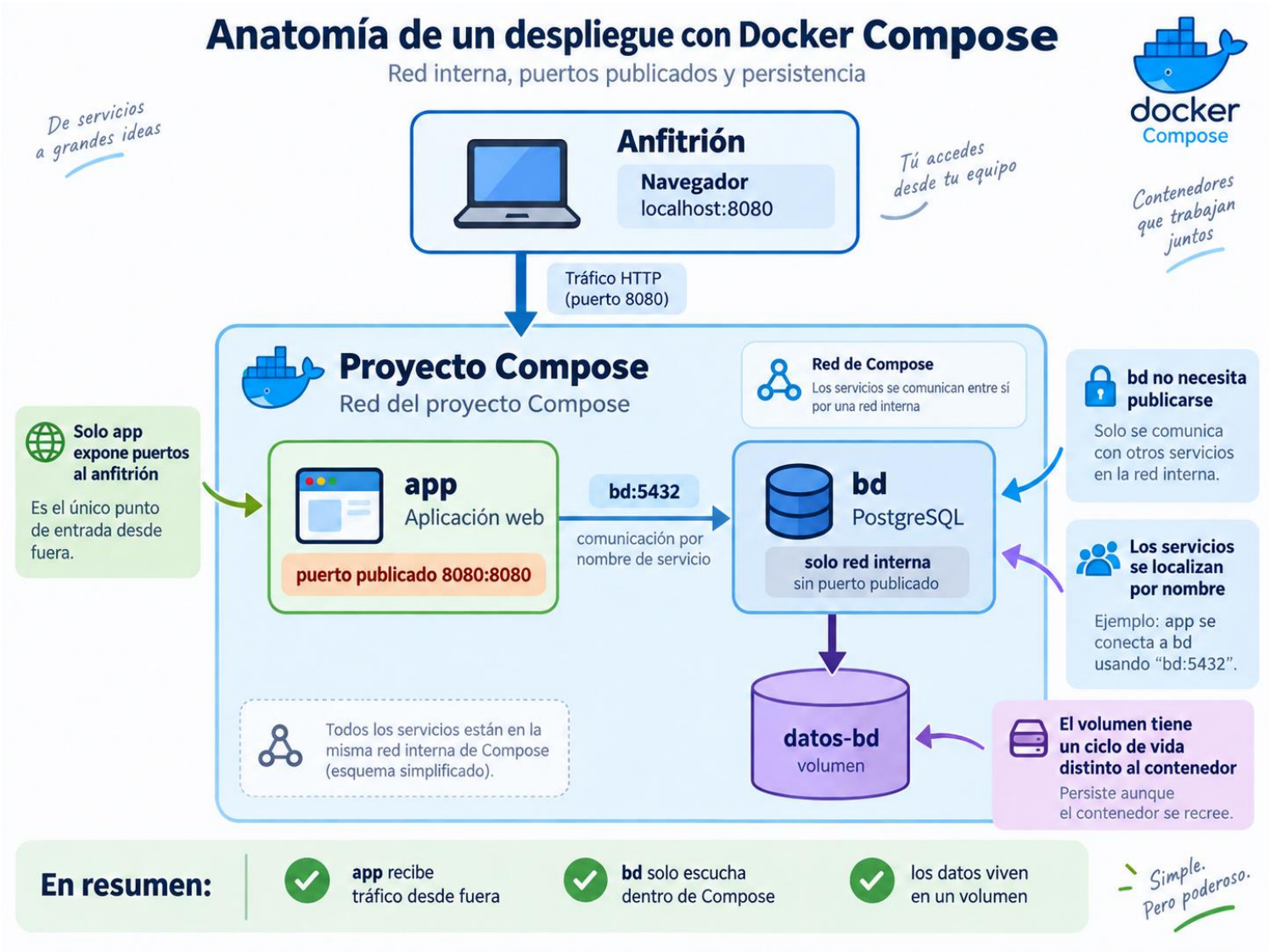
compose.yaml

```
services:
  bd:
    image: postgres:18-alpine
  app:
    image: mi-app:1.0.0
```

Fragmento inicial: faltan configuración, almacenamiento y dependencias.
mi-app:1.0.0 representa una imagen propia, disponible localmente o publicada.

docker compose up -d crea o actualiza los recursos descritos.

Arquitectura del proyecto Compose



Comunicación entre servicios y puertos publicados

Dentro del proyecto

app utiliza bd:5432.

El nombre de servicio permanece como referencia aunque cambie la IP.

PostgreSQL no necesita publicar su puerto.

Desde el anfitrión

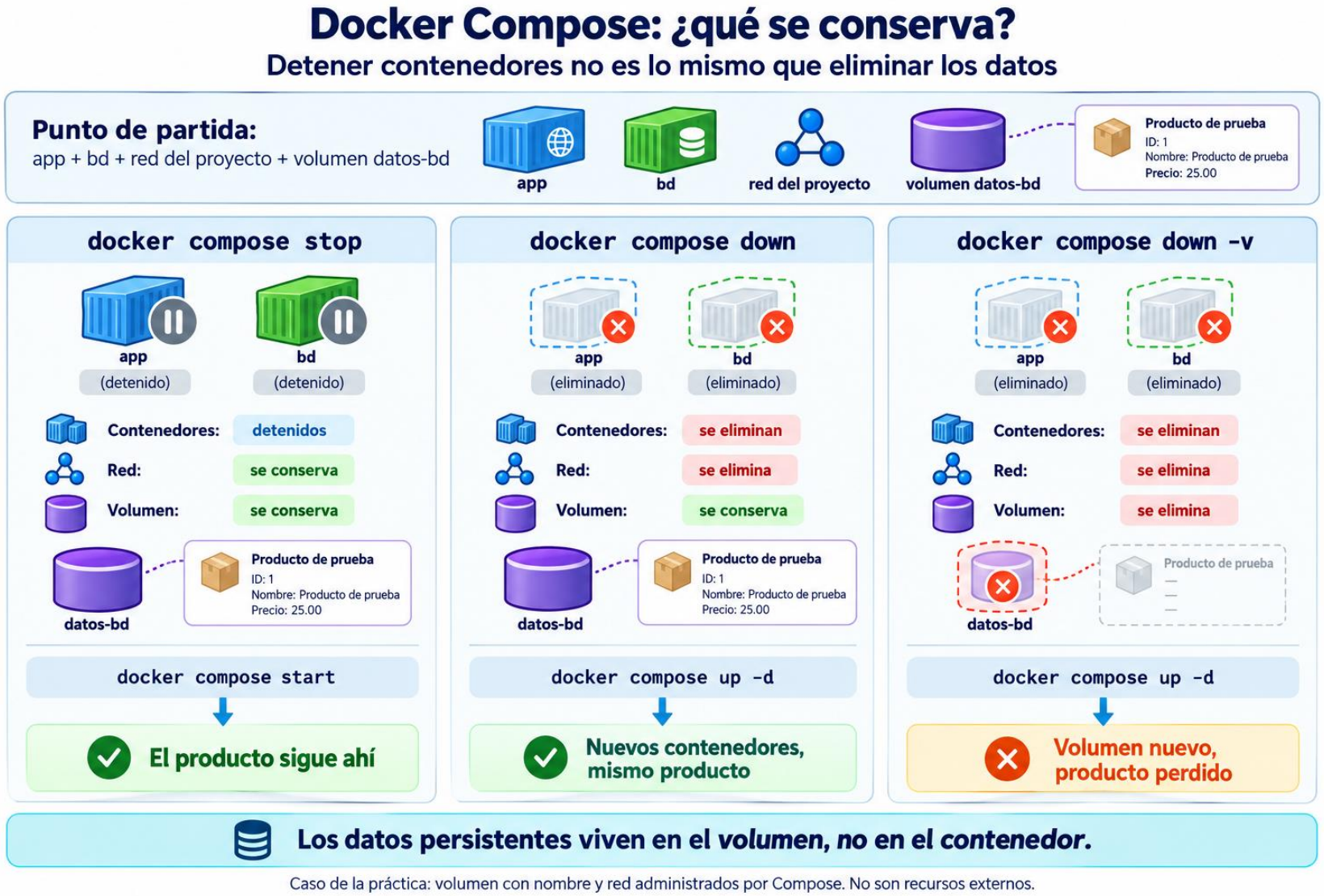
El navegador accede por localhost:8080.

8080:8080 relaciona puerto del anfitrión y puerto del contenedor.

Publicamos únicamente app.

EXPOSE documenta un puerto. ports lo publica hacia el anfitrión.

Persistencia al detener o recrear contenedores



Declaración del volumen de PostgreSQL

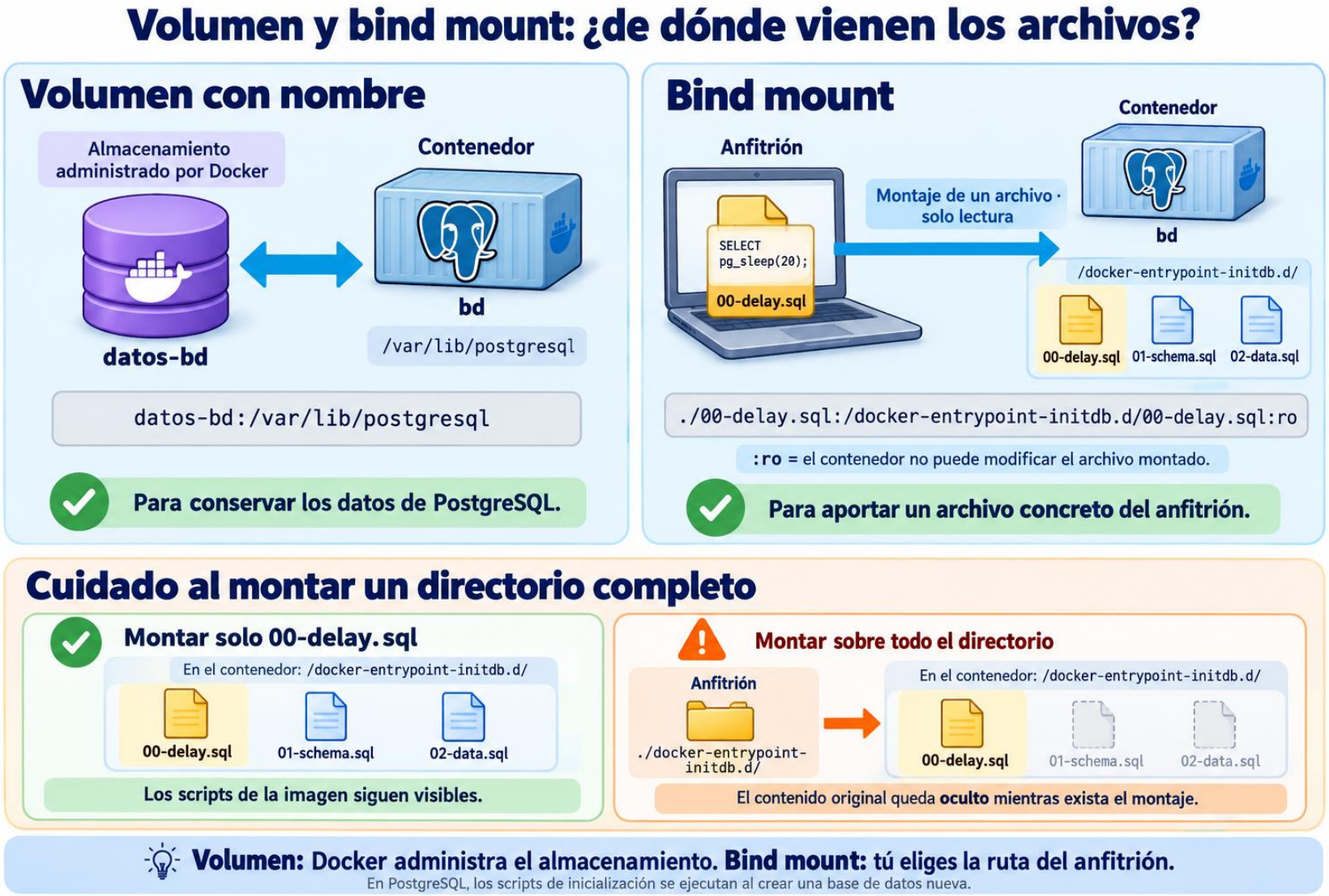
```
services:
  bd:
    volumes:
      - datos-bd:/var/lib/postgresql

volumes:
  datos-bd:
```

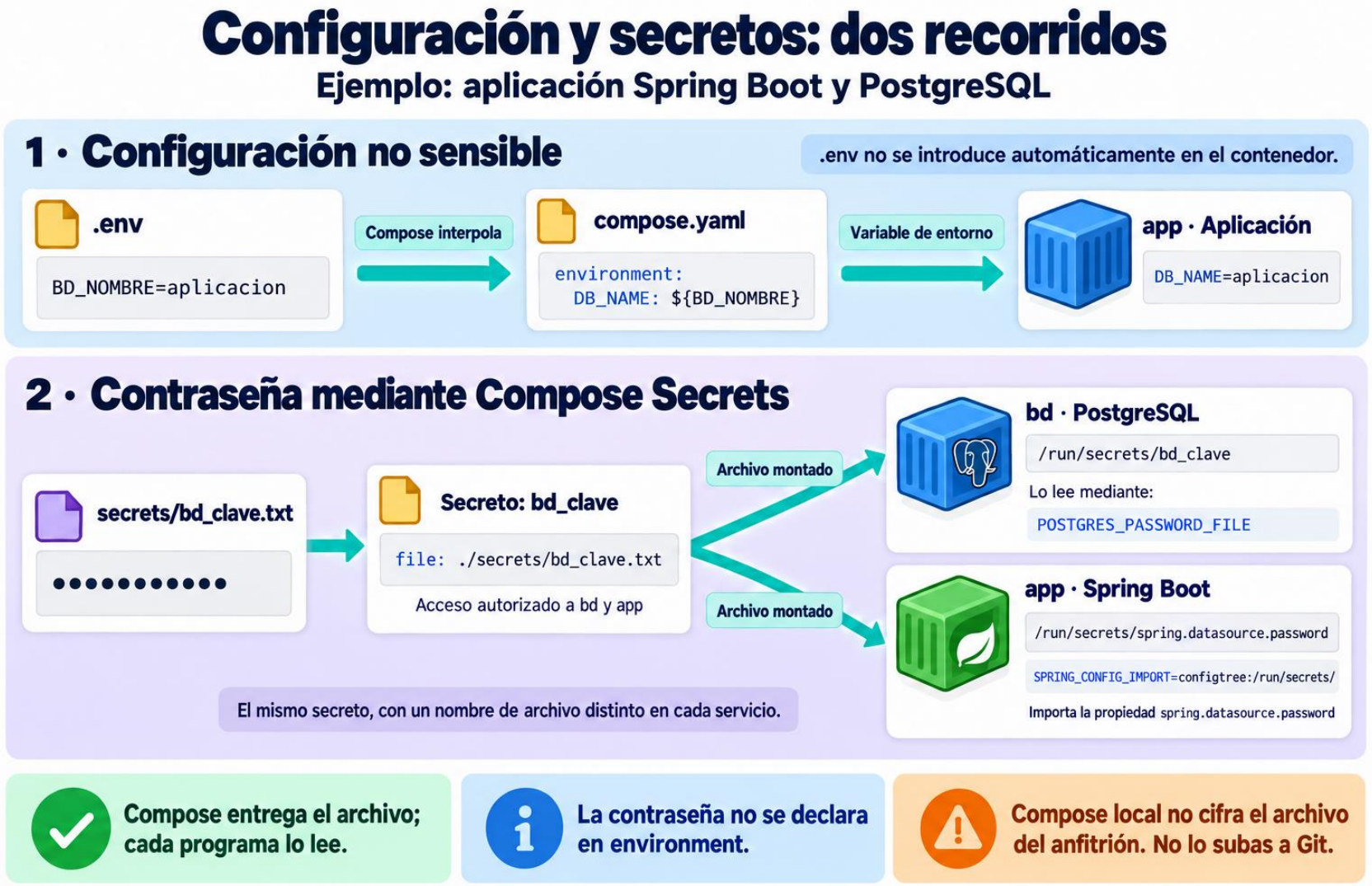
PostgreSQL 18 utiliza /var/lib/postgresql.
No copies automáticamente la ruta habitual de versiones anteriores.

El servicio monta el volumen. La sección superior volumes lo declara.

Volúmenes y archivos del anfitrión



Configuración externa y Compose Secrets



Archivos versionados y archivos locales

Se versionan

compose.yaml

.env.example

secrets/bd_clave.example.txt

README y reglas de .gitignore

Se preparan en cada equipo

.env: valores locales no sensibles.

secrets/bd_clave.txt: solo la contraseña, sin nombre de variable.

.gitignore no retira archivos que ya estaban registrados.

Los ejemplos contienen valores ficticios. Los archivos locales quedan fuera de Git.

Lectura del secreto en PostgreSQL

```
services:
  bd:
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/bd_clave
    secrets:
      - bd_clave

secrets:
  bd_clave:
    file: ./secrets/bd_clave.txt
```

El archivo local se monta solo en los servicios autorizados.

La imagen de PostgreSQL implementa POSTGRES_PASSWORD_FILE.

Ejemplo: secreto en Spring Boot

```
services:
  app:
    environment:
      SPRING_CONFIG_IMPORT: "configtree:/run/secrets/"
    secrets:
      - source: bd_clave
        target: spring.datasource.password
```

source identifica el secreto. target define el nombre del archivo.
La ruta resultante es /run/secrets/spring.datasource.password.

Spring Boot importa el archivo como la propiedad spring.datasource.password.

Alcance y límites de Compose Secrets

Qué aporta

Entrega la contraseña como archivo.

Autoriza el acceso por servicio.

Evita declarar el valor de la contraseña en environment.

Qué debemos comprobar

Permisos de lectura del usuario de app.

Protección del archivo local: Compose no lo cifra.

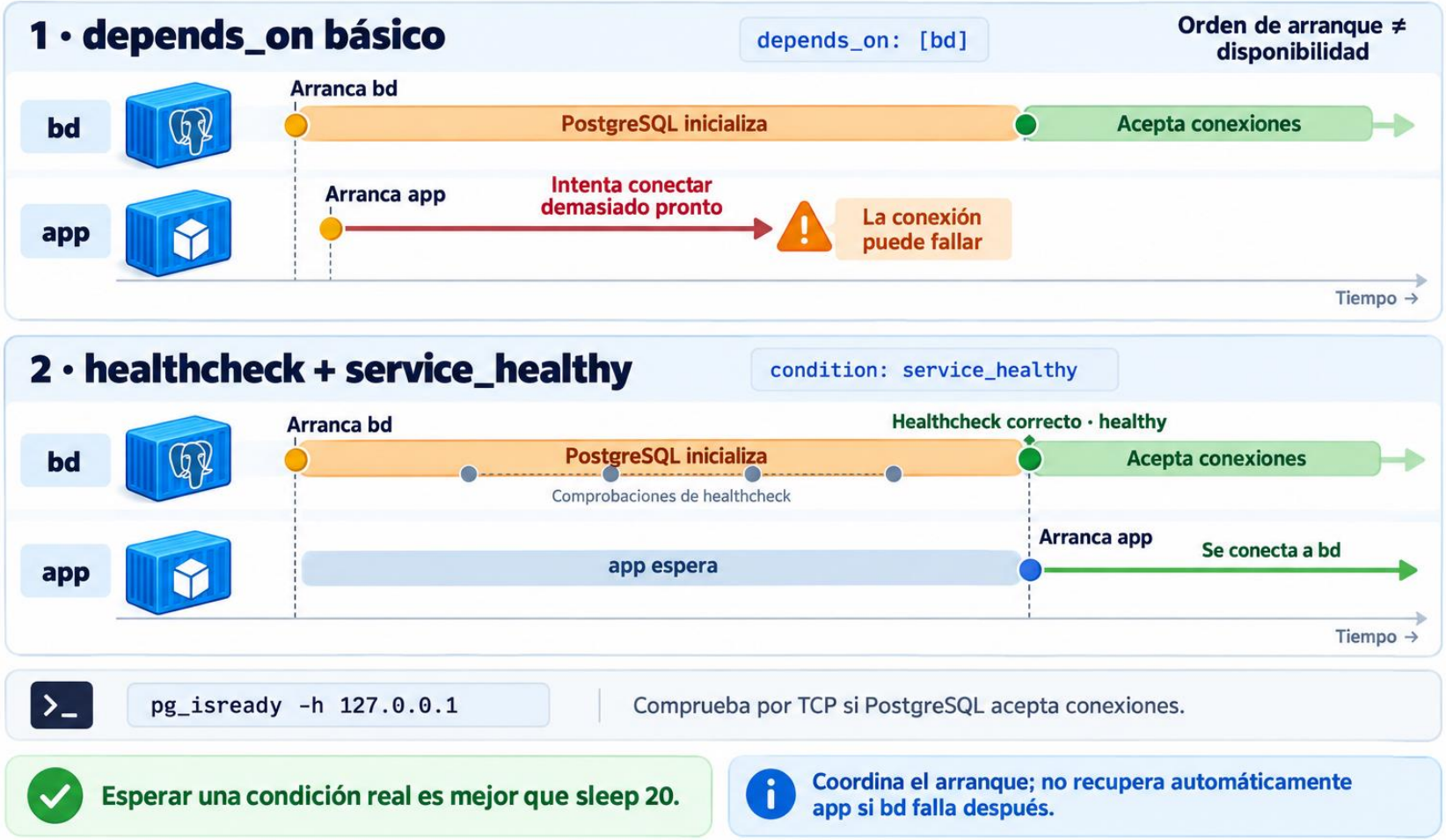
Coordinación de cambios de contraseña con PostgreSQL.

Cambiar el archivo no modifica la contraseña ya guardada en PostgreSQL.

Dependencias de arranque y disponibilidad

Arrancado no significa preparado

depends_on ordena el arranque; service_healthy espera una comprobación




```
pg_isready -h 127.0.0.1
```

Comprueba por TCP si PostgreSQL acepta conexiones.

Esperar una condición real es mejor que sleep 20.

Coordina el arranque; no recupera automáticamente app si bd falla después.

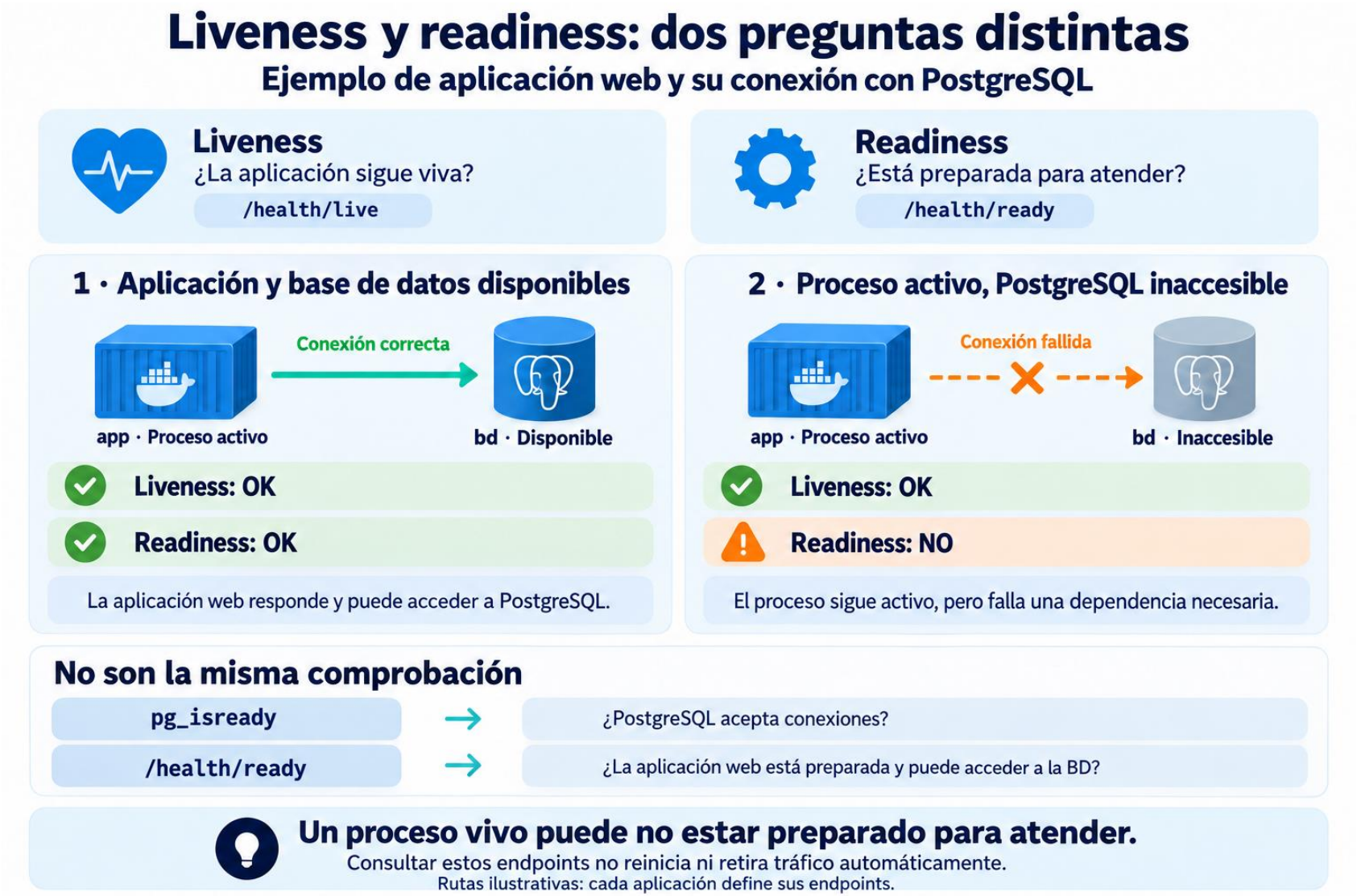
Healthcheck de PostgreSQL

```
healthcheck:
  test:
    - CMD-SHELL
    - pg_isready -h 127.0.0.1 -U "$${POSTGRES_USER}" -d "$${POSTGRES_DB}"
  interval: 2s
  timeout: 2s
  retries: 30
  start_period: 10s
```

127.0.0.1 exige conexión TCP.
$$ conserva el signo $ para la evaluación dentro del contenedor.

pg_isready comprueba si PostgreSQL acepta conexiones.

Liveness y readiness de la aplicación



Validación de la configuración y de la ejecución

Antes de arrancar

docker compose config --quiet

Comprueba que Compose puede interpretar la configuración.

Los ejemplos permiten preparar un clon limpio.

Con el sistema arrancado

Comprobar lectura del secreto.

Consultar su endpoint de readiness.

Probar una función de la aplicación.

Recrear contenedores y comprobar los datos.

Interpretar el YAML no demuestra que la aplicación pueda conectar.

Operación y diagnóstico del conjunto

```
docker compose up -d          Crear o actualizar
docker compose ps -a          Consultar el estado
docker compose logs app       Investigar app
docker compose logs bd        Investigar PostgreSQL
docker compose config         Ver configuración
docker compose exec app ...   Comprobar dentro
docker compose stop           Detener
docker compose start          Volver a arrancar
docker compose down           Eliminar contenedores
```

El estado y los logs orientan la investigación antes de editar el YAML.

Validación automatizada del despliegue

Validación de configuración

Preparar archivos desde ejemplos.

Comprobar que existen los archivos necesarios.

Interpretar el Compose en CI.

Prueba de ejecución

Arrancar los servicios.

Verificar lectura del secreto y conexión.

Probar disponibilidad y persistencia.

Validar el YAML y probar la ejecución comprueban problemas diferentes.

Conexión con la actividad 2.3

Configuración del conjunto

Diagnosticar la comunicación.

Comprobar persistencia y puertos.

Separar configuración y secretos.

Disponibilidad y entrega

Observar el tiempo de inicialización.

Configurar healthcheck y dependencia.

Documentar y validar el despliegue.

Aplicaremos el despliegue de dos servicios al proyecto Escaparate.

Alcance de Compose y cierre del tema

Qué queda descrito

Servicios e imágenes publicadas.

Comunicación y puertos.

Persistencia y secretos.

Dependencias de arranque.

Dónde termina

Despliegue sobre un motor Docker.

No distribuye por sí solo la carga entre nodos ni reemplaza un nodo caído.

La operación requiere diagnóstico y comprobaciones.

Siguiente tema: Nginx como punto de entrada delante de app.